

Modul - Fortgeschrittene Programmierkonzepte

Bachelor Informatik

01 - Einführung

Prof. Dr. Marcel Tilly

Fakultät für Informatik, Cloud Computing

Tiny logo competition!



Let's try to find a new logo and replace the boring one...



Price



- Send your proposals to me until Friday!
- Voting starts Saturday ...
- ... until Monday eol (=end of lecture)!

Organizational Things

Material:

- Lectures
 - Learning Campus: <https://learning-campus.th-rosenheim.de/course/view.php?id=1482>
 - Online: <https://matworx.org/index.html?inf-fpk/README>
- GitLab:
 - Examples: https://inf-git.fh-rosenheim.de/inf-fpk/hsro-inf-fpk-github-io/-/tree/master/examples?ref_type=heads
 - Exercises: <https://inf-git.fh-rosenheim.de/inf-fpk>

IMPORTANT: Slides, Script, Exercises ...in English! ☺

- Subscribe under **Fortgeschrittene Programmierkonzepte (INF-B3), WiSe24/25**
(Selbsteinschreibung ohne Schlüssel!)

Lecture - Timetable

- **Date:** Every Monday, 09:45- 11:15 in R0.03
- **Exercises** in S1.29: Monday, 3./4./5.
 - Group 1: 11:45-13:15 (in presence)
 - Group 2: 13:45-15:15 (in presence)
 - **LB Elvis Helbrandt**
- **Group selection in LC** (now open!)
- Communication via Discord (**#fpk**)

Klausur!

- schriftliche Prüfung (SP, 90 Minuten)
- am Ende des Semesters
- erlaubt ist ein Buch mit ISBN Nummer
- Anmeldung über OSC
- Was kommt dran?
 - Alles was in der Vorlesung dran war!
 - Coding on paper!

I would like to work with you on small coding projects!

- Form a team (2-4 members)
- Create **your own idea** and drive your **own** project
- Improve your coding skills
- Collaborate with **me**
- Use GitLab or Github
- Show at the end your result (if you want!)

Ideas: Footballmanager, lightweight editor, notes, TicTacToe, Trainingsplaner ...

Aus dem Modulhandbuch

Die Studierenden ...

- ... **vertiefen** ihre Kenntnisse in der objektorientierten Programmierung am Beispiel einer geeigneten Programmiersprache (hier: Java!)
- ... können die Möglichkeiten und Gefahren der objektorientierten Programmierung **beurteilen**.
- ... **sind befähigt**, alle wichtigen Programmierkonzepte für das Programmieren im Großen im Sinne der Komponentenorientierung anzuwenden.
- ... **erarbeiten sich die Grundlagen** der funktionalen Programmierung und deren Anwendungsgebiete.

Programmieren 1

- Imperative programming in C
- Constants, Variables, Expressions, Functions, I/O
- Data structures (fields, arrays, lists)
- Pointer ☺

Programmieren 1

- Imperative programming in C
- Constants, Variables, Expressions, Functions, I/O
- Data structures (fields, arrays, lists)
- Pointer ☺

Programmieren 2 (OOP)

- Objekt-oriented Programming (OOP) in Java
- Classes and Objects
- Interfaces and Inheritance
- Exception handling

Agenda of FPK

[Learning Campus](#)

1. Classes, Interfaces and Inheritance
2. Generics
3. Reflection and Annotations
4. GUI
5. Design Pattern and UML
6. Parallel Processing
7. Functional Programming and Streams

Agenda für heute

1. *Inform:*

Use your trusted advisors: [chatGPT](#) -- [Google](#) -- [SO](#) -- [Java Docs](#) -- [Translate](#)

2. *Memorize:*

The git version control system (<https://git-scm.com/>)

3. *Automate:*

The Gradle build tool (<https://gradle.org/>)

4. *Organize:*

The IntelliJ IDEA (<https://www.jetbrains.com/idea/>)

5. *(Optional) Collaborate*

Practice cross-repository pull requests and learn about *continuous integration*

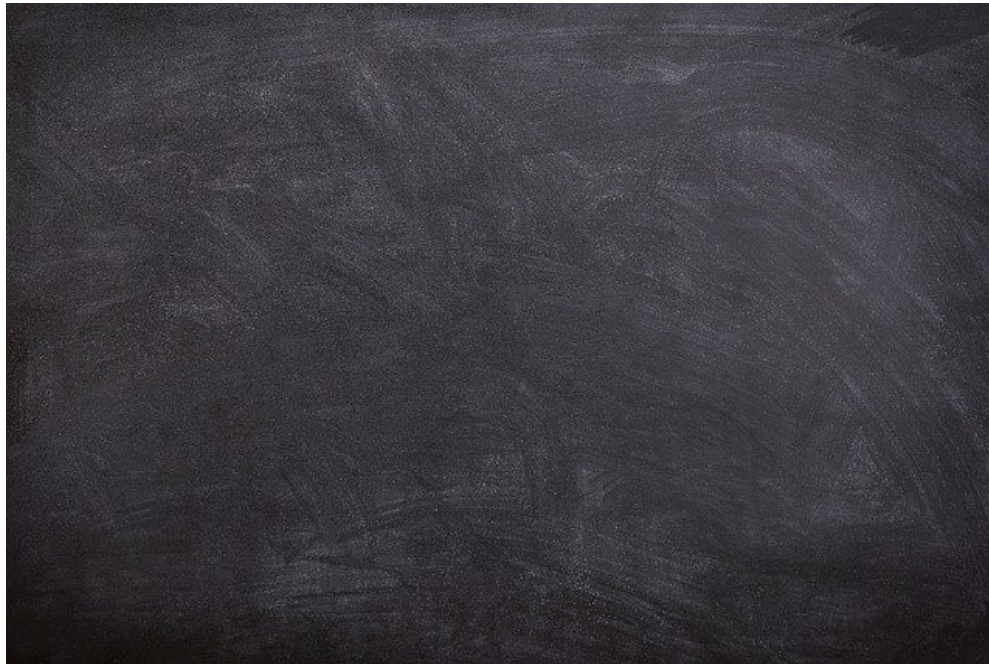
Code with me in IntelliJ

SO = stackoverflow

Exercise 1



Which version control systems do you know?



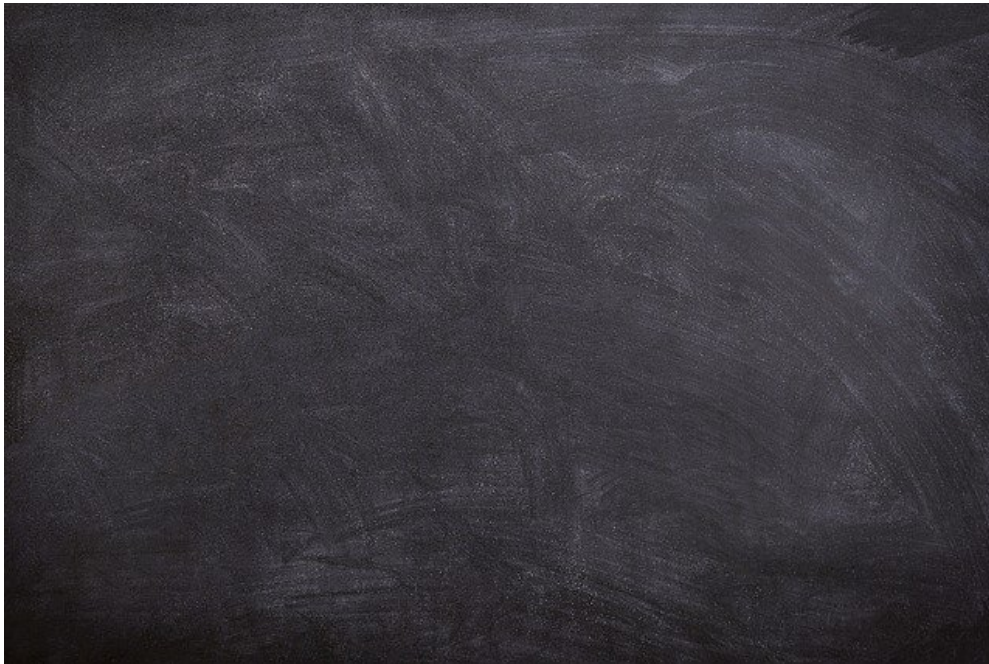
Git

- Git is a distributed version-control system for tracking changes in source code during software development.
- It is designed for coordinating work among programmers, but it can be used to track changes in any set of files.
- [Git](#) is the *de-facto* state of the art [version control system](#).
 - Some of you might remember [CVS \(concurrent versions system\)](#) or [subversion](#).
 - Generally speaking, you should always use a version control system (VCS) when working on code, so you can keep track of changes.
- **Print and laminate:** <https://education.github.com/git-cheat-sheet-education.pdf>
- For the more visual: <http://ndpsoftware.com/git-cheatsheet.html>
- If you run into a mess (and you will): <https://gandrille.github.io/tech-notes/GIT/git-pretty.pdf>

Exercise 2



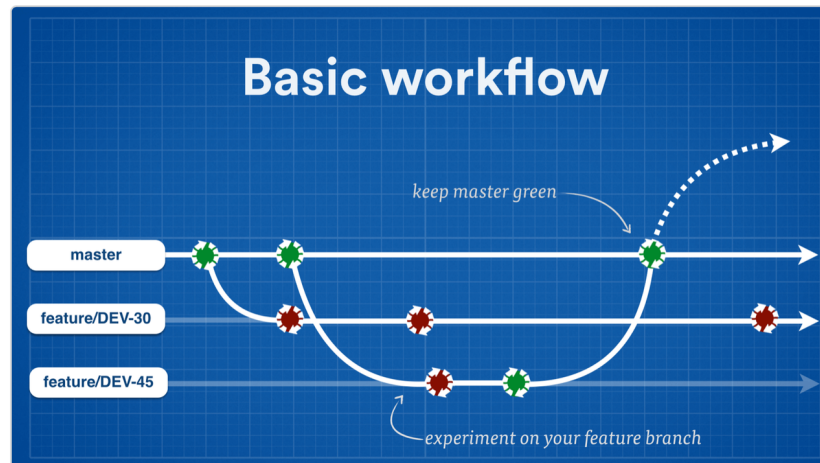
Why is versioning so important?



Git and feature branches



<https://www.atlassian.com/continuous-delivery/continuous-delivery-workflows-with-feature-branching-and-gitflow>



- Git Guide: <https://rogerdudler.github.io/git-guide/>
- Git != GitHub

The Gradle Build Tool (GBT)

Gradle is an open-source build-automation system that builds upon the concepts of Apache Ant and Apache Maven and introduces a Groovy-based domain-specific language instead of the XML form used by Apache Maven for declaring the project configuration.



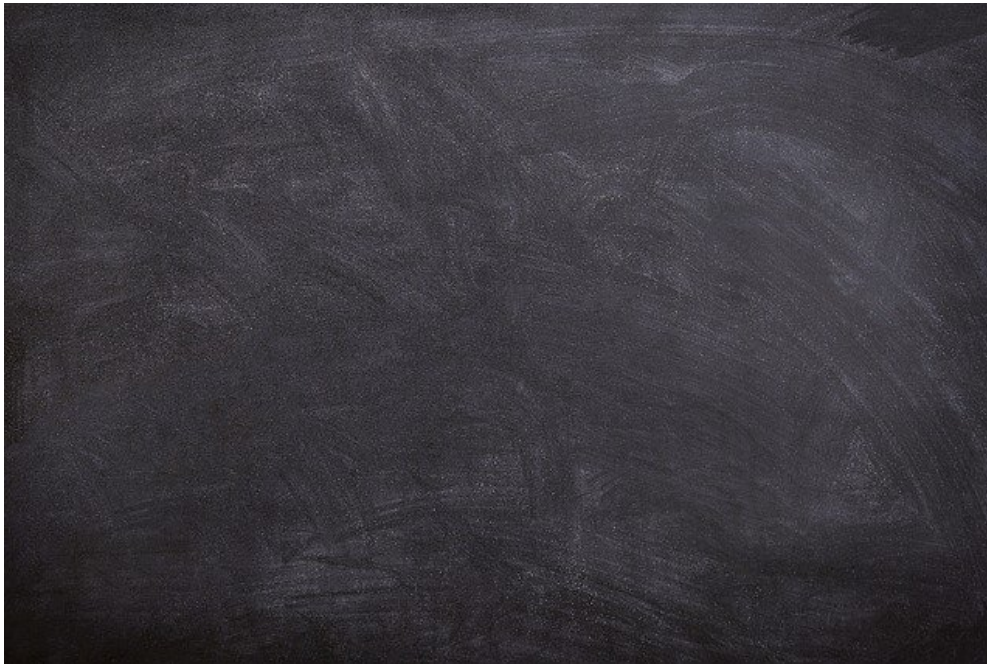
<https://gradle.org>

- `gradle init --type java-application` to bootstrap a project
- `./gradlew build` to use the [Gradle wrapper](https://gradle.org) to be independent of locally installed Gradle
- apply plugin: 'eclipse' and `./gradlew eclipse` to generate Eclipse project files
- apply plugin: 'idea' and `./gradlew idea` to generate IntelliJ files (note: these are file-based project descriptions, not the new directory based `.idea/*`)

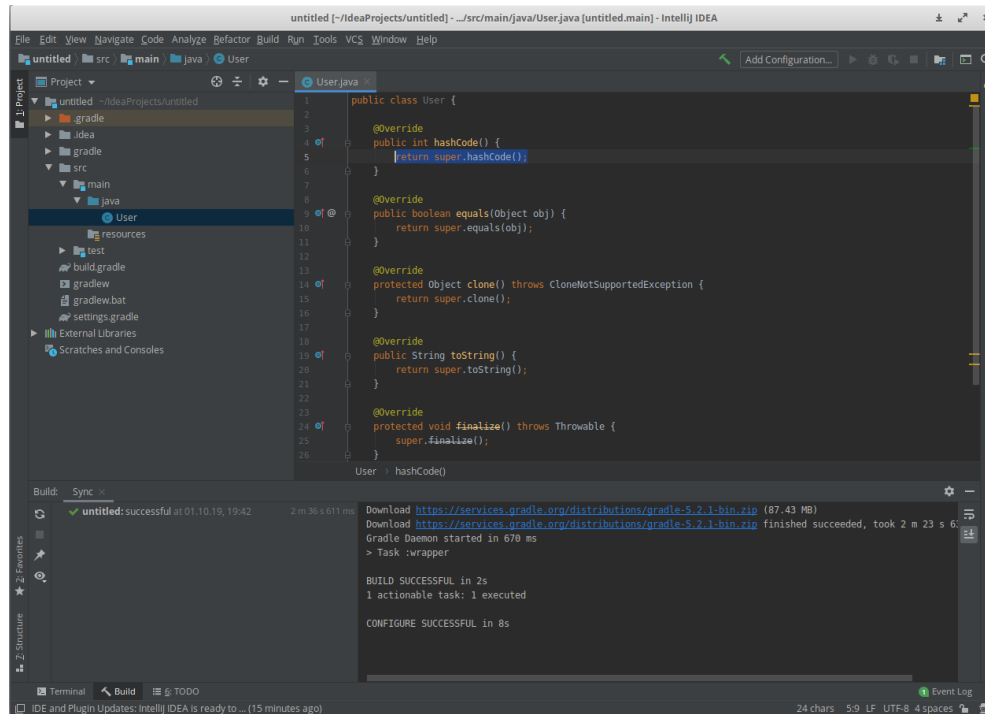
Exercise 3



Why is automation so important?



<https://www.jetbrains.com/idea/>

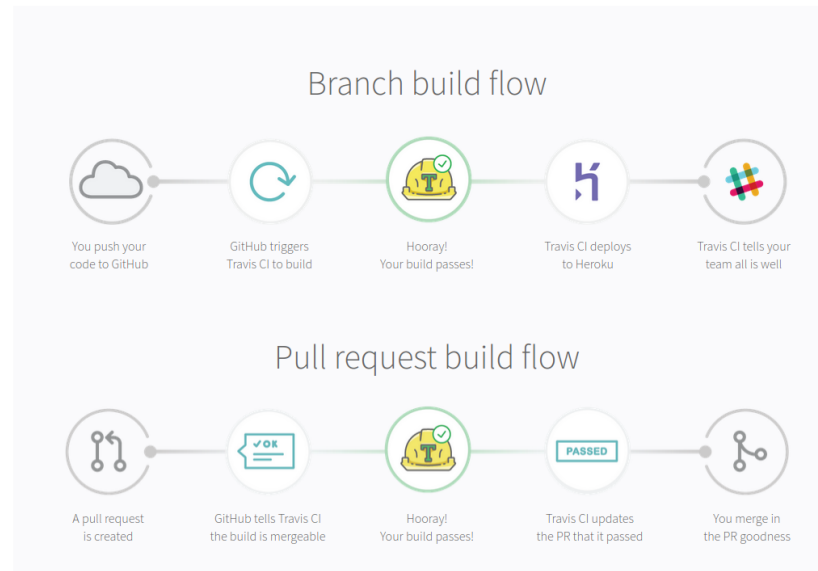


- Collaboration means splitting the work
- Teamwork means working together
- Use feature branches and automated tests (JUnit)
- Use automated build and test runner

=> CI/CD

- CI and CD are two acronyms that are often mentioned when people talk about modern development practices.
- CI is straightforward and stands for **continuous integration**: A practice that focuses on making preparing a release easier.
- CD can either mean **continuous delivery** or **continuous deployment**.

Travis CI is a hosted continuous integration service used to build and test software projects hosted at GitHub. Travis CI provides various paid plan for private projects, and a free plan for open source. (<https://travis-ci.org/>)





- An open source automation server
- Jenkins provides hundreds of plugins to support building, deploying and automating any project.
- <https://www.jenkins.io/>



- GitLab is a web-based DevOps lifecycle tool that provides a Git-repository manager providing wiki, issue-tracking and continuous integration and deployment pipeline features, using an open-source license, developed by GitLab Inc.
- <https://about.gitlab.com/> and I am using it for the lecture: <https://inf-git.fh-rosenheim.de/inf-fpk>

.gitlab-ci.yml:

```
image: gradle:8.3.0-jdk20

stages:
  - build

before_script:
  - echo pwd # debug

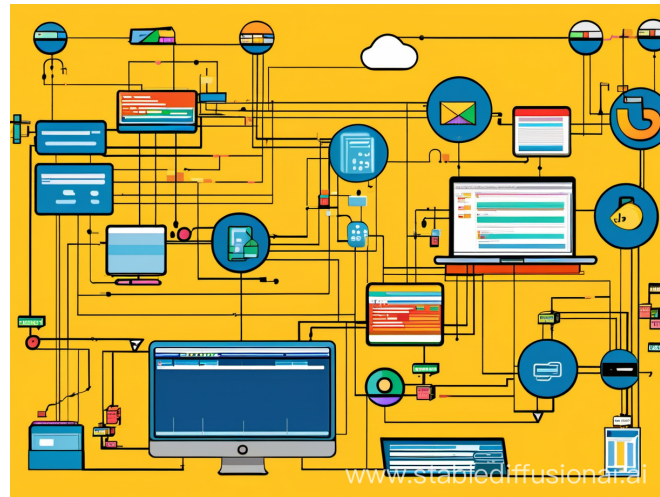
build:
  stage: build
  script:
    - ./gradlew test

after_script:
  - echo "End CI"
```

Comparison

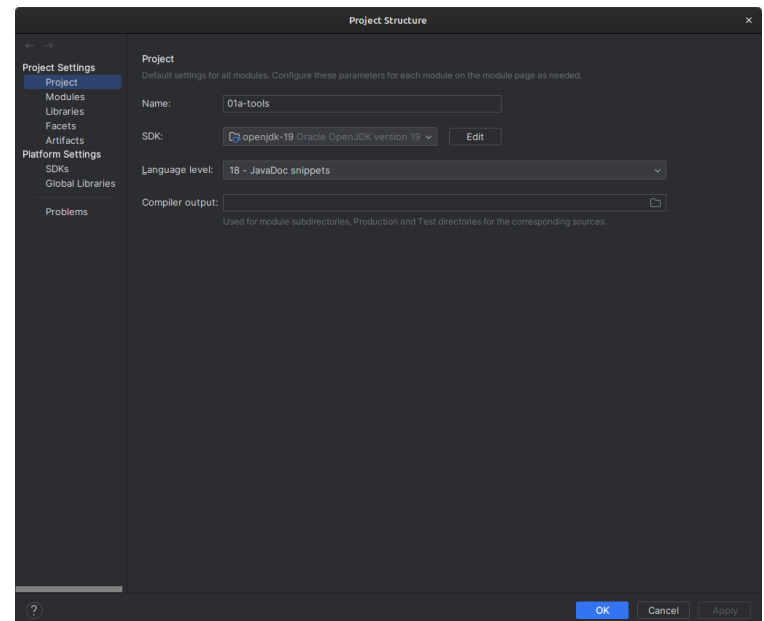


- <https://stackshare.io/stackups/gitlab-ci-vs-jenkins-vs-travis-ci#pros>
- We are going to use **GitLab**!
- We (TH Rosenheim) are hosting our own server: <https://inf-git.fh-rosenheim.de>
- Ensure that you have access!



Which versions do we use?

- Java/JDK: [JDK 19.0.2](#) or [openJDK 19](#)
- Java Version: Java 18 (JavaDoc Snippets)
- Gradle: 8.3
- Git: 2.42.0



Summary



- We will look into advanced programming concepts in Java (starting next week!)
- We will move towards professional software engineering tools (starting today!)
 - Git
 - IntelliJ Idea
 - Gradle
 - GitLab
- Let's try to have fun!



Final thought!

